

CSE 341 Spring 2026 Quizzes

Questions only - ordered by source filenames Quiz1, Quiz2, ..., Quiz10

Quiz 1

PART A: Answer on the front side: which one is better? Detecting the errors during compile time or run time. Discuss the advantages and disadvantages.

PART B: Answer on the backside: Give examples of orthogonality features and non-orthogonality features in C.

Quiz 2

PART A: Answer on the front side: Java has both compilation and just-in-time compilation. Explain the difference.

PART B: Answer on the backside: Given the C expression below, show the lexical and syntax analysis steps.

`X = Y * 3 + X;`

Quiz 3

PART A: Answer on the front side: Why was the slowness of interpretation of programs acceptable in the early 1950s? Why did it become unacceptable in the mid-1950s?

PART B: Answer on the backside: Give an example of a typeless programming language. What are the arguments both for and against the idea of a typeless language?

Quiz 4

PART A: Write a grammar for the language consisting of strings that have n copies of the letter `a` followed by the same number of copies of the letter `b`, where $n > 0$. For example, the strings `ab`, `aaaabbbb`, and `aaaaaaaaabbbbbbbbb` are in the language but `a`, `abb`, `ba`, and `aaabb` are not.

PART B: Rewrite the BNF below (1) to add the prefix `++` and `--` unary operators of C (`++a`) and (2) to make `+` operator right associative. `++` and `--` have the highest precedence.

BNF:

```
<expr>  -> <expr> + <term> | <expr> - <term> | <term>
<term>  -> <term> * <factor> | <term> / <factor> | <factor>
<factor> -> <exp> ** <factor> | <exp>
<exp>   -> (<expr>) | id
```

Quiz 5

PART A: Given the BNF below, write the attribute grammar for a language where data types cannot be mixed in expressions, but assignment statements need not have matching types on both sides.

```
<assign> -> <var> = <expr>
<expr>   -> <var>[2] + <var>[3] | <var>
<var>    -> A | B | C
```

Hint for second rule:

```
<expr> -> <var>[2] + <var>[3]
Semantic rule: <expr>.actualType <- ...
Predicate: ...
```

PART B: Compute the weakest precondition for each of the following assignment statements and postconditions:

```
a = 2 * (b - 1) - 1    {a > 0}
b = (c + 10) / 3      {b > 6}
a = a + 2 * b - 1     {a > 1}
x = 2 * y + x - 1     {x > 11}
```

Quiz 6

PART A: Show a trace of the recursive descent parser for the string $a * (b + c)$ with the following EBNF.

```
<expr>   -> <term> {(+ | -) <term>}
<term>   -> <factor> {( * | / ) <factor>}
<factor> -> id | int_constant | (<expr>)
```

PART B: Draw a state transition diagram for a lexical analyzer that recognizes three token types: identifiers (IDENT), integer literals (INT_LIT), and real number literals (REAL_LIT, e.g. 3.14). Your diagram should use character classes (letter, digit) for transitions. Show clearly the start state, intermediate states, accepting states with their corresponding token types, and how a dot (.) distinguishes an integer from a real number.

Quiz 7

PART A: The following grammar has indirect left recursion. Show that it is indirectly left-recursive by expanding the rules step by step. Then eliminate all left recursion so the grammar is suitable for a recursive-descent parser.

```
A -> B a | c
B -> A b | d
```

PART B: Using the LR parsing table below, show the bottom-up parse of $\text{id} * \text{id} + \text{id}$. For each step show: stack, input, and action.

(1) $E \rightarrow E + T$ (2) $E \rightarrow T$ (3) $T \rightarrow T * F$ (4) $T \rightarrow F$ (5) $F \rightarrow \text{id}$

State	id	+	*	\$	E	T	F
0	s5				1	2	3
1		s6		acc			
2		r2	s7	r2			
3		r4	r4	r4			
5		r5	r5	r5			
6	s5					9	3
7	s5						10
9		r1	s7	r1			
10		r3	r3	r3			

Quiz 8

PART A: Consider the following program, written in JavaScript-like syntax:

```
// main program
var x, y, z;

function sub1() {
    var a, y, z;
    ...
}

function sub2() {
    var a, b, z;
    ...
}

function sub3() {
    var a, x, w;
    ...
}
```

Given the following calling sequences, what variables are visible during execution of the last subprogram activated? Include with each visible variable the name of the unit where it is declared.

```
main calls sub1;
sub1 calls sub2;
sub2 calls sub3.
```

Answer this question two times: one for static scoping and one for dynamic scoping.

PART B: What is a static ancestor of a subprogram? What is a dynamic ancestor of a subprogram?

Quiz 9

PART A: Consider the following two-dimensional array declaration in a language that uses row-major order and zero-based indexing:

```
double matrix[4][6];
```

Assume that the base address of the array is 2000 and that each double element occupies 8 bytes.

(a) Write the general access function for computing the address of `matrix[i][j]`.

(b) Compute the address of `matrix[2][3]`.

(c) A programmer writes the following C code:

```
double *ptr = (double *) matrix;
```

What is the value of `*(ptr + 15)`? Which element of the matrix does it correspond to? Show your work using the relationship between pointer arithmetic and row-major layout.

PART B: C and C++ are not considered strongly typed languages, while ML and F# are. Explain why, making specific reference to the language features that cause C/C++ to fall short of the strong typing requirement. Then explain how coercion can weaken the value of strong typing even in languages like Java that are considered nearly strongly typed - give a concrete example.

Quiz 10

PART A: Answer on the front side.

Consider the following C expression, where a, b, c, and d are integer variables:

```
a = b + c * b - d / b++ + (a > b)
```

(i) Rewrite the expression fully parenthesized to show the exact order of operator evaluation according to C's precedence and associativity rules.

(ii) Identify two distinct language design hazards present in this expression and explain why each is a hazard.

PART B: Answer on the back side.

Define short-circuit evaluation of Boolean expressions.

Give one concrete example in C where short-circuit evaluation prevents a run-time error, and give one concrete example where short-circuit evaluation causes a subtle bug (i.e., a side effect that is silently skipped). Explain both cases clearly.